Git Branches: Features unabhängig entwickeln und mit Git verwalten

TL;DR
Die Commits in Git bauen aufeinander auf und bilden dabei eine verkettete "Liste". Diese "Liste" nennt man auch *Branch* (Entwicklungszweig). Beim Initialisieren eines Repositories wird automatisch ein Default-Branch angelegt, auf dem die Commits dann eingefügt werden.
Weitere Branches kann man mit `git branch` anlegen, und die Workingcopy kann mit `git switch` oder `git checkout` auf einen anderen Branch umgeschaltet werden. Auf diese Weise kann man an mehreren Features parallel arbeiten, ohne dass die Arbeiten sich gegenseitig stören.
Zum Mergen (Vereinigen) von Branches gibt es `git merge`. Dabei werden die Änderungen im angegebenen Branch in den aktuell in der Workingcopy ausgecheckten Branch integriert und hier ggf. ein neuer Merge-Commit erzeugt. Falls es in beiden Branches inkompatible Änderungen an der selben Stelle gab, entsteht beim Mergen ein Merge-Konflikt. Dabei zeigt Git in den betroffenen Dateien jeweils an, welche Änderung aus welchem Branch stammt und man muss diesen Konflikt durch Editieren der Stellen manuell beheben.
Mit `git rebase` kann die Wurzel eines Branches an eine andere Stelle verschoben werden. Dies wird später bei Workflows eine Rolle spielen.

Videos

- VL Git Branches
- Demo Anlegen und Mergen von Branches
- Demo Auflösen von Merge-Konflikten
- Demo HEAD

Unterlagen

- PDF-Version

Neues Feature entwickeln/ausprobieren

```
A---B---C  master
```

- Bisher nur lineare Entwicklung: Commits bauen aufeinander auf (lineare Folge von Commits)
- `master` ist der (Default-) Hauptentwicklungszweig
  - Pointer auf letzten Commit
  - Default-Name: "`master`" (muss aber nicht so sein bzw. kann geändert werden)

Anmerkung: Git und auch Github haben den Namen für den Default-Branch von `master` auf `main`geändert. Der Name an sich ist aber für Git bedeutungslos und kann mittels `git config --global init.defaultBranch <name>` geändert werden. In Github hat der Default-Branch eine gewisse Bedeutung, beispielsweise ist der Default-Branch das automatische Ziel beim Anlegen von Pull-Requests. In Github kann man den Default-Namen global in den User-Einstellungen (Abschnitt "Repositories") und für jedes einzelne Repository in den Repo-Einstellungen (Abschnitt "Branches") ändern.

Entwicklung des neuen Features soll stabilen `master`-Branch nicht beeinflussen => Eigenen Entwicklungszweig für die Entwicklung des Features anlegen:

1. Neuen Branch erstellen: `git branch wuppie`
2. Neuen Branch auschecken: `git checkout wuppie` oder `git switch wuppie`

Alternativ: `git checkout -b wuppie` oder `git switch -c wuppie` (neuer Branch und auschecken in einem Schritt)

```
A---B---C  master, wuppie
```

Startpunkt: prinzipiell beliebig (jeder Commit in der Historie möglich).

Die gezeigten Beispiel zweigen den neuen Branch direkt vom aktuell ausgecheckten Commit/Branch ab. Also aufpassen, was gerade in der Workingcopy los ist!

Alternativ nutzen Sie die Langform: `git branch wuppie master` (mit `master` als Startpunkt; hier kann jeder beliebige Branch, Tag oder Commit genutzt werden).

Nach Anlegen des neuen Branches zeigen beide Pointer auf den selben Commit.

Anmerkung: In neueren Git-Versionen wurde der Befehl "**`switch`**" eingeführt, mit dem Sie in der Workingcopy auf einen anderen Branch wechseln können. Der bisherige Befehl "`checkout`" funktioniert aber weiterhin.

Arbeiten im Entwicklungszweig ...

```
          D  wuppie
         /
A---B---C  master
```

- Entwicklung des neuen Features erfolgt im eigenen Branch: beeinflusst den stabilen `master`-Branch nicht
- Wenn in der Workingcopy der Feature-Branch ausgecheckt ist, gehen die Commits in den Feature-Branch; der `master` bleibt auf dem alten Stand
- Wenn der `master` ausgecheckt wäre, würden die Änderungen in den `master` gehen, d.h. der `master` würde sich ab Commit `C` parallel zu `wuppie` entwickeln

Problem: Fehler im ausgelieferten Produkt

```
          D  wuppie
         /
A---B---C  master
```

Fix für `master` nötig:

1. `git checkout master`

2. `git checkout -b fix`
3. Änderungen in `fix` vornehmen …

Das führt zu dieser Situation:

```
          D  wuppie
         /
A---B---C  master
         \
          E  fix
```

`git checkout <branchname>` holt den aktuellen Stand des jeweiligen Branches in die Workingcopy. (Das geht in neueren Git-Versionen auch mit `git switch <branchname>`.)

Man kann weitere Branches anlegen, d.h. hier im Beispiel ein neuer Feature-Branch `fix`, der auf dem `master` basiert. Analog könnte man auch Branches auf der Basis von `wuppie` anlegen …

Fix ist stabil: Integration in *master*

```
          D  wuppie
         /
A---B---C  master
         \
          E  fix
```

1. `git checkout master`
2. `git merge fix` => **fast forward** von `master`
3. `git branch -d fix`

Der letzte Schritt entfernt den Branch `fix`.

```
          D  wuppie
         /
A---B---C---E  master
```

- Allgemein: `git merge <branchname>` führt die Änderungen im angegebenen Branch `<branchname>` in den aktuell in der Workingcopy ausgecheckten Branch ein. Daraus resultiert für den aktuell ausgecheckten Branch ein neuer Commit, der Branch `<branchname>` bleibt dagegen auf seinem bisherigen Stand.

  Beispiel:

  - Die Workingcopy ist auf `A`
  - `git merge B` führt `A` und `B` zusammen: `B` wird **in** `A` gemergt
  - Wichtig: Der Merge-Commit (sofern nötig) findet hierbei in `A` statt!

In der Abbildung ist `A` der `master` und `B` der `fix`.

- Nach dem Merge existieren beide Branches weiter (sofern sie nicht explizit gelöscht werden)
- Hier im Beispiel findet ein sogenannter "Fast forward" statt.

"Fast forward" ist ein günstiger Spezialfall beim Merge: Beide Branches liegen in einer direkten Kette, d.h. der Zielbranch kann einfach "weitergeschaltet" werden. Ein Merge-Commit ist in diesem Fall nicht notwendig und wird auch nicht angelegt.

Feature weiter entwickeln ...

```
          D---F  wuppie
         /
A---B---C---E  master
```

1. `git switch wuppie`
2. Weitere Änderungen im Branch `wuppie` ...

`git switch <branchname>` holt den aktuellen Stand des jeweiligen Branches in die Workingcopy. Man kann also jederzeit in der Workingcopy die Branches wechseln und entsprechend weiterarbeiten.

Hinweis: Während der neue `git switch`-Befehl nur Branches umschalten kann, funktioniert `git checkout` sowohl mit Branchnamen und Dateinamen - damit kann man also auch eine andere Version einer Datei in der Workingcopy "auschecken". Falls gleiche Branch- und Dateinamen existieren, muss man für das Auschecken einer Datei noch "`--`" nutzen: `git checkout -- <dateiname>`.

Feature ist stabil: Integration in *master*

```
          D---F  wuppie                            D---F  wuppie
         /                    =>                  /     \
A---B---C---E  master                    A---B---C---E---G  master
```

1. `git checkout master`
2. `git merge wuppie`
   => Kein *fast forward* möglich: Git sucht nach gemeinsamen Vorgänger

Hier im Beispiel ist der Standardfall beim Mergen dargestellt: Die beiden Branches liegen nicht in einer direkten Kette von Commits, d.h. hier wurde parallel weitergearbeitet.

Git sucht in diesem Fall nach dem gemeinsamen Vorgänger beider Branches und führt die jeweiligen Änderungen (Differenzen) seit diesem Vorgänger in einem Merge-Commit zusammen.

Im `master` entsteht ein neuer Commit, da kein *fast forward* beim Zusammenführen der Branches möglich!

Anmerkung: `git checkout wuppie; git merge master` würde den `master` in den `wuppie` mergen, d.h. der Merge-Commit wäre dann in `wuppie`.

Beachten Sie dabei die “Merge-Richtung”:

- Die Workingcopy ist auf A
- `git merge B` führt A und B zusammen: B wird **in** A gemergt
- Wichtig: Der Merge-Commit (sofern nötig) findet hierbei in A statt!

In der Abbildung ist A der `master` und B der `wuppie`.

Achtung: Richtung beachten! `git checkout A; git merge B` führt beide Branches zusammen, genauer: führt die Änderungen von B in A ein, d.h. der entsprechende Merge-Commit ist in A!

Konflikte beim Mergen

(Parallele) Änderungen an selber Stelle => Merge-Konflikte

```
$ git merge wuppie
Auto-merging hero.java
CONFLICT (content): Merge conflict in hero.java
Automatic merge failed; fix conflicts and then commit the result.
```

Git fügt Konflikt-Marker in die Datei ein:

```
<<<<<<< HEAD:hero.java
public void getActiveAnimation() {
    return null;
=======
public Animation getActiveAnimation() {
    return this.idleAnimation;
>>>>>>> wuppie:hero.java
```

- Der Teil mit HEAD ist aus dem aktuellen Branch in der Workingcopy
- Der Teil aus dem zu mergenden Branch ist unter `wuppie` notiert
- Das `=======` trennt beide Bereiche

Merge-Konflikte auflösen

Manuelles Editieren nötig (Auflösung des Konflikts):

1) Entfernen der Marker
2) Hinzufügen der Datei zum Index
3) Analog für restliche Dateien mit Konflikt
4) Commit zum Abschließen des Merge-Vorgangs

Alternativ: Nutzung graphischer Oberflächen mittels `git mergetool`

Konsole: Branchen und Mergen (https://youtu.be/B8sesK1GyiE)

Rebasen: Verschieben von Branches

```
          D---F  wuppie                          D---F  wuppie
         /                    =>                /     \
A---B---C---E  master                   A---B---C---E---G  master
```

Bisher haben wir Branches durch Mergen zusammengeführt. Dabei entsteht in der Regel ein extra Merge-Commit (im Beispiel `G`), außer es handelt sich um ein *fast forward*. Außerdem erkennt man in der Historie sehr gut, dass hier in einem separaten Branch gearbeitet wurde, der irgendwann in den `master` gemergt wurde.

Leider wird dieses Vorgehen in großen Projekten recht schnell sehr unübersichtlich. Außerdem werden Merges in der Regeln nur von besonders berechtigten Personen (Manager) durchgeführt, die im Falle von Merge-Konflikten diese dann selbst auflösen müssten (ohne aber die fachliche Befähigung zu haben). Hier greift man dann häufig zur Alternative *Rebase*. Dabei wird der Ursprung eines Branches auf einen bestimmten Commit verschoben. Im Anschluss ist dann ein Merge mit *fast forward*, also ohne die typischen rautenförmigen Ketten in der Historie und ohne extra Merge-Commit möglich. Dies kann aber auch als Nachteil gesehen werden, da man in der Historie den früheren Branch nicht mehr erkennt! Ein weiterer schwerwiegender Nachteil ist, dass alle Commits im verschobenen Branch umgeschrieben werden und damit neue Commit-IDs bekommen. Das verursacht bei der Zusammenarbeit in Projekten massive Probleme! Als Vorteil gilt, dass man mögliche Merge-Konflikte bereits beim Rebasen auflösen muss, d.h. hier muss derjenige, der den Merge "beantragt", durch einen vorherigen Rebase den konfliktfreien Merge sicherstellen. Mehr dazu in "Branching-Strategien" und "Workflows".

```
git rebase master wuppie
```

führt zu

```
               D'---F'  wuppie
              /
A---B---C---E  master
```

Nach dem Rebase von `wuppie` auf `master` sieht es so aus, als ob der Branch `wuppie` eben erst vom `master` abgezweigt wurde. Damit ist dann ein *fast forward* Merge von `wuppie` in den `master` möglich, d.h. es gibt keine Raute und auch keinen extra Merge-Commit (hier nicht gezeigt).

Man beachte aber die Änderung der Commit-IDs von `wuppie`: Aus `D` wird `D'`! (Datum, Ersteller und Message bleiben aber erhalten.)

Don't lose your HEAD

- Branches sind wie Zeiger auf letzten Stand (Commit) eines Zweiges
- `HEAD`: Spezieller Pointer
  - Zeigt auf den aktuellen Branch der Workingcopy

- Früheren Commit auschecken (ohne Branch): “headless state”
  - Workingcopy ist auf früherem Commit
  - Kein Branch => Änderungen gehen verloren!

    Eventuelle Änderungen würden ganz normal als Commits auf dem HEAD-Branch aufgezeichnet. Sobald man aber einen anderen Branch auscheckt, wird der HEAD auf diesen anderen Branch gesetzt, so dass die eben gemachten Commits “in der Luft hängen”. Sofern man die SHA’s kennt, kommt man noch auf die Commits zurück. Allerdings laufen von Zeit zu Zeit interne Aufräum-Aktionen, so dass die Chance gut steht, dass die “kopflosen” Commits irgendwann tatsächlich verschwinden.

Wrap-Up

- Anlegen von Branches mit `git branch`
- Umschalten der Workingcopy auf anderen Branch: `git checkout` oder `git switch`
- Mergen von Branches und Auflösen von Konflikten: `git merge`
- Verschieben von Branches mit `git rebase`

Zum Nachlesen

- Chacon und Straub (2014, Kap. 3)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k3: Erzeugen von Branches
- k3: Mergen von Branches, Auflösen möglicher Konflikte
- k3: Rebasen von Branches

Challenges

Branches und Merges

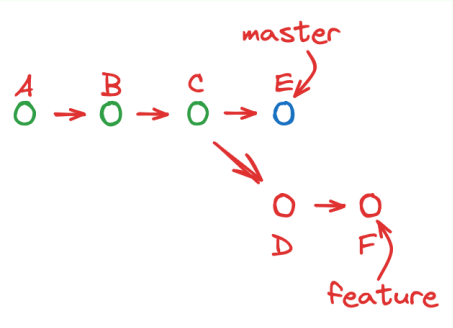
1. Legen Sie in Ihrem Projekt einen Branch an. Ändern Sie einige Dateien und committen Sie die Änderungen. Checken Sie den Master-Branch aus und mergen Sie die Änderungen. Was beobachten Sie?
2. Legen Sie einen weiteren Branch an. Ändern Sie einige Dateien und committen Sie die Änderungen. Checken Sie den Master-Branch aus und ändern Sie dort ebenfalls:
   - Ändern Sie eine Datei an einer Stelle, die nicht bereits im Branch modifiziert wurde.
   - Ändern Sie eine Datei an einer Stelle, die bereits im Branch manipuliert wurde.

   Committen Sie die Änderungen.

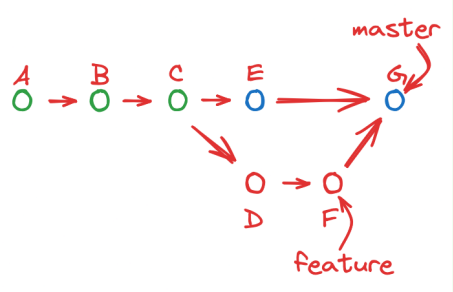
   Mergen Sie den Branch jetzt in den Master-Branch. Was beobachten Sie? Wie lösen Sie Konflikte auf?

Mergen am Beispiel

Sie verwalten Ihr Projekt mit Git. Es existieren zwei Branches: `master` (zeigt auf Commit C) und `feature` (zeigt auf Version F). In Ihrer Workingcopy haben Sie den Branch `feature` ausgecheckt:



(1) Mit welcher Befehlsfolge können Sie den Branch `feature` in den Branch `master` mergen, so dass nach dem Merge die im folgenden Bild dargestellte Situation entsteht?



(Der Merge läuft ohne Konflikte ab. Es ist irrelevant, welcher Branch am Ende in der Workingcopy ausgecheckt ist.)

(2) Wie können Sie erreichen, dass es keinen Merge-Commit gibt, sondern dass die Änderungen in D und F im `master` als eine lineare Folge von Commits erscheinen?

Interaktive Git-Tutorials: Schaffen Sie die Rätsel?

- Learn Git Branching
- Oh My Git!
- Git Time
- Tutorial: A Visual Git Reference

Quellen

Atlassian Pty Ltd. 2022. „Become a Git Guru.“ 2022. https://www.atlassian.com/git/tutorials.
Chacon, S., und B. Straub. 2014. *Pro Git*. 2. Aufl. Apress. https://git-scm.com/book/en/v2.
Github Inc. 2022. „Git Cheat Sheets“. 2022. https://training.github.com/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** ba2c853 (Lecture: remove 'quizzes' from the metadata to avoid any confusion with the new lecture quizzes, 2025-04-09)